Project Milestone 1: Verifiable Tasks

Fall 2026: COMP 480/580

Due: October 31, 2026

1 Introduction

AI agents can use tools to perform tasks such as retrieving information, processing data, and creating files. However, an agent's statement that it has completed a task does not establish that its result is correct. To reliably evaluate an agent, we need a precise task specification, a reproducible environment, and an executable verifier that checks the actual result. In this milestone, you will create **5** original tasks, each as its own plugin with an execution environment and a verifier that checks its outcome. Build and test each plugin locally, then publish it on RUSH Lab, our open-source platform where agents can run tasks and be evaluated.

The GitHub repository is at `https://github.com/RUSH-LAB/nanoagent`. Ask your AI to tell you more about it!

Optional: If anything in the repository is buggy, missing, unclear, or could be better, open an issue on GitHub or contact the course staff. The repository is under active development, and it would be fun if we could improve it together over the semester.

2 Tasks

Here is a list of components what a task should include:

- **Task:** Define the task's inputs, tools, outputs, and success conditions.
- **Environment:** Include data, dependencies, and reproducible setup instructions.
- **Verifier:** Check actual results. Test correct, incorrect, missing, malformed, and boundary cases. **The verifier must not call an LLM as judge.** You may use an LLM to help write it, but the verifier itself must be a program that runs without one.
- **Validation and publication:** Test, run, and publish the task. Provide reproducible evidence.

3 Instructions

3.1 Set up your plugin

Install Git and uv. Create each of your 5 plugins as its own repository in your GitHub account. For a worked example, see the Weather Alert plugin by the TAs. Use it as a reference, not a template: each of your tasks must be original. The nanoagent repository documents what a plugin needs, and you may use AI tools to help you work out the requirements. Our agent harness follows one principle: **structured input and structured output**. Every task and every tool declares a typed input and a typed output in `contract.py`, so each value the agent receives or produces has a fixed shape that code can check. From each plugin folder, run the tests and fix every error before you publish. Try to make the most of your coding agents to help you write the code!

3.2 Publish and validate on RUSH Lab

Follow these steps to publish your plugin and validate it on RUSH Lab:

1. **Register.** Add your GitHub username to the Google Sheet. The TAs use it to invite you to RUSH Lab.
2. **Install RUSH Lab.** Sign in at `https://rush-lab.org` with the GitHub account you registered. A pop-up asks you to install RUSH Lab on your computer: copy the command it shows and run it in a terminal (on Windows, use WSL). Contact the TAs as soon as possible if you have trouble installing.
3. **AI model.** The agent uses `DeepSeek V4 Flash 0731`, which cannot currently be changed. By default it runs on an OpenRouter API key will be provided by the TAs, capped at $10 per month. To use your own key instead, add it under **Settings** → **AI**.
4. **Publish.** Submit each tested plugin to the RUSH Lab marketplace. If the checks report errors, fix them and resubmit.

Note: the marketplace checks only that your plugin is packaged correctly. Passing them does not mean your task or verifier is correct; that is what you are graded on.

4 What to Submit

1. **Plugins.** The GitHub URL of each of your 5 plugins.
2. **Report.** One PDF that answers, for each plugin:
   (a) Why is the task objectively verifiable?
   (b) How is the environment reproducible?
   (c) How does the verifier handle incorrect outputs and boundary cases?
   (d) What limitations remain?

   Include the task inputs you ran, the test and validation logs, and the verifier's verdicts. Put this evidence in the report, not in the plugin repositories.

The TAs should be able to reproduce your results without any manual fixes.